RAG

Project Structure & Code

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

RAG Project Structure Code

1. Project Structure

```
rag-project/
├── data/
│   └── documents/      # Your PDF, TXT, DOCX files here
├── app/
│   ├── ingestion.py    # Load and read documents
│   ├── chunking.py     # Split text into chunks
│   ├── embeddings.py   # Create embeddings
│   ├── vector_store.py # Store in vector DB (Qdrant)
│   ├── retriever.py    # Retrieve relevant chunks
│   ├── llm.py          # Call LLM (Ollama/OpenAI)
│   └── rag_pipeline.py # Orchestrate the flow
├── config/
│   └── settings.py     # Load env and config
├── main.py             # Entry point
├── requirements.txt    # Python dependencies
└── .env                # Environment variables
```

2. config/settings.py

```python
# config/settings.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()  # Load variables from .env

@dataclass
class Settings:
    OPENAI_API_KEY: str = os.getenv("OPENAI_API_KEY", "")
    OLLAMA_BASE_URL: str = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
    QDRANT_URL: str = os.getenv("QDRANT_URL", "http://localhost:6333")
    QDRANT_COLLECTION: str = os.getenv("QDRANT_COLLECTION", "rag_docs")
    CHUNK_SIZE: int = int(os.getenv("CHUNK_SIZE", "500"))
    CHUNK_OVERLAP: int = int(os.getenv("CHUNK_OVERLAP", "50"))
    EMBEDDING_MODEL: str = os.getenv("EMBEDDING_MODEL", "all-MiniLM-L6-v2")
    LLM_MODEL: str = os.getenv("LLM_MODEL", "llama3")

settings = Settings()
```

Explanation:

- Loads environment variables from .env file.
- Uses dataclass for easy configuration.
- Provides default values for local development.
- We can access using: from config.settings import settings

3. app/ingestion.py

```python
# app/ingestion.py
from pathlik import Path
from langchain_community.document_laoders import PyPDFLoader, \
    TextLoader, Docx2txtLoader

def load_documents(data_path: str = "data/documents"):
    documents = []
    path = Path(data_path)
    for file in path.glob("*.pdf"):
        loader = PyPDFLoader(str(file))
        documents.extend(loader.load())
    for file in path.glob("*.txt"):
        loader = TextLaader(str(file))
        documents.extend(loader.load())
    for file in path.glob("*.docx"):
        loader = Docx2txtLoader(str(file))
        documents.extend(loader.load())
    return documents
```

Explanation:

- Reads PDF, TXT and DOCX files from data/documents folder.
- Uses LangChain document loaders.
- Returns a list of Document objects with content and metadata.

4. app/chunking.py

```python
# app/chunking.py
from langchain.text_splitter import RecursiveCharacterTextSplitter

def split_documents(documents, chunk_size:int = 500, chunk_overlap: int = 50):
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", " ", ""]
    )
    chunks = text_splitter.split_documents(documents)
    return chunks
```

Explanation:

- Uses long documents into smaller chunks.
- Uses RecursiveCharacterTextSplitter for better context handling.
- Default_model: all-MiniLM-L6-v2 help maintain context between chunks.

5. app/embeddings.py

```python
# app/embeddings.py
from langchain_community.embeddings import HuggingFaceEmbeddings
from config.settings import settings

def get_embeddings():
    return HuggingFaceEmbeddings(
        model_name=settings.EMBEDDING_MODEL,
        model_kwargs={"device": "cpu"}, # use "cuda" if GPU available)
    )
```

Explanation:

- Uses HuggingFace embedding model.
- Converts text chunks into vector embeddings.
- Default model: all-MiniLM-L6-v2 (fast and good quality).

6. app/vector_store.py

```python
# app/vector_store.py
from qdrant_client import QdrantClient
from qdrant_client.http.models import Distance, VectorParams
from config.settings import settings

def create_vector_store(embeddings):
    client = QdrantClient(url=settings.QDRANT_URL)
    collection_name = settings.QDRANT_COLLECTION
    client.recreate_collection(
        collection_name=collection_name,
        vectors_config=VectorParams(
            size=384,  # for all-MiniLM-L6-v2
            distance=Distance.COSINE,
        ),
    )
    return client, collection_name
```

Explanation:

- Creates a Qdrant collection to store embeddings.
- Uses cosine similarity for search.
- Returns Qdrant client and collection name.

7. app/retriever.py

```python
# app/retriever.py
from langchain_qdrant import Qdrant
from config.settings import settings

def get_retriever(client, collection_name, embeddings, k: int = 4):
    vector_storer = Qdrant(
        client=client,
        collection_name=collection_name,
        embeddings=embeddings,
    )
    return vector_store.as_retriever(search_kwargs={"k": k})
```

Explanation:

- Creates a retriever from Qdrant vector store.
- Uses similarity search to get top-k relevant chunks.
- Returns a LangChain retriever.

8. app/llm.py

```python
# app/llm.py
from langchain_community.llms import Ollama
from langchain_openai import ChatOpenAI
from config.settings import settings

def get_llm():
    if settings.OPENAI_API_KEY:
        return ChatOpenAI(
            model_name="gpt-3.5-turbo",
            temperature=0.2,
        )
    else:
        return Ollama(
            model=settings.LLM_MODEL,
            base_url=settings.OLLAMA_BASE_URL,
            temperature=0.2,
        )
```

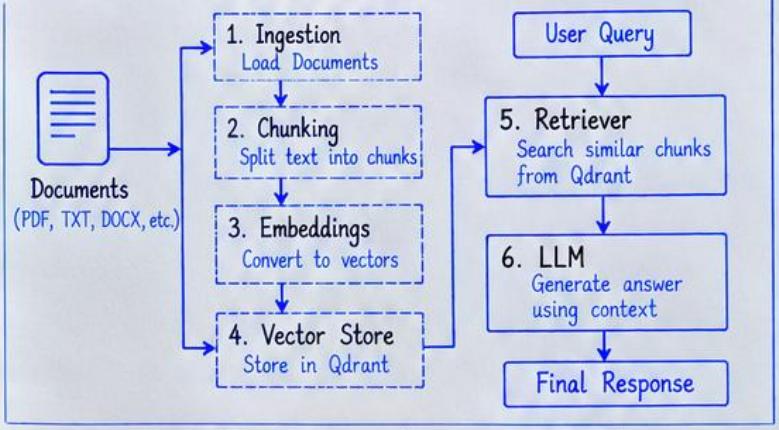
Explanation:

- Uses OpenAI if API key is provided.
- Otherwise uses Ollama for local LLM.
- Temperature is kept low for accurate answers.

Rahul Yadav

RAG Project - Complete Architecture

1. System Architecture Overview



2. Data Flow in RAG

1. Load documents from data/documents/ folder (PDF, TXT, DOCX, etc.)
2. Split documents into smaller chunks.
3. Convert each chunk into vector embeddings.
4. Store embeddings in Qdrant vector database.
5. User asks a question (query).
6. Retrieve top-k relevant chunks from Qdrant.
7. Pass retrieved chunks + query to LLM.
8. LLM generates a final answer using the context.
9. Return the response to the user.

3. Required Python Packages (requirements.txt)

```
langchain>=0.2.0
langchain-community>=0.2.0
qdrant-client>=1.7.0
sentence-transformers>=2.6.0
pypdf>=4.0.0
python-docx>=1.1.0
python-dotenv>=1.0.0
openai>=1.40.0      # if using OpenAI
ollama>=0.1.7       # if using Ollama
fastapi>=0.115.0    # optional (for API)
uvicorn>=0.30.0     # optional (for API)
streamlit>=1.36.0   # optional (for UI)
```

Explanation:

- langchain: Core framework for RAG.
- langchain-community: Community integrations (loaders, vector stores, etc.).
- qdrant-client: Python client for Qdrant DB.
- sentence-transformers: Convert text to embeddings.
- pypdf: Read PDF files.
- python-docx: Read DOCX files.
- python-dotenv: Load environment variables.
- openai / ollama: Use any LLM (OpenAI or local).
- fastapi, streamlit: Optional for API or UI.

4. .env (Environment Variables)

```
# OpenAI Configuration (use if you want OpenAI)
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxx
# OR Ollama Configuration (use if you want local LLM)
LLM_MODEL=llama3
OLLAMA_BASE_URL=http://localhost:11434
# Qdrant Configuration
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=rag_docs
# Chunking Settings
CHUNK_SIZE=500
CHUNK_OVERLAP=50
# Embedding Model
EMBEDDING_MODEL=all-MiniLM-L6-v2
# Other Settings
TOP_K=4
TEMPERATURE=0.2
```

5. How All Modules Work Together ?

1. `1agestion.py` → Loads documents from data/ folder
2. `chunking.py` → Splits documents into chunks
3. `embeddings.py` → Creates vector embeddings for chunks
4. `vector_store.py` → Stores embeddings in Qdrant
5. `retriever.py` → Retrieves relevant chunks for a query
6. `llm.py` → Calls LLM (OpenAI / Ollama)
7. `rag_pipeline.py` → Orchestrates the complete RAG flow
8. `main.py` → Entry point to run the project
9. `config/settings.py` → Loads environment variables and configs

6. Running the Project (Step by Step)

1. Create a new folder and add all files as shown.
2. Create a virtual environment (optional).

```
python -m venv venv
source venv/bin/activate    # on Mac/Linux
venv\Scripts\activate       # on Windows
```

3. Install dependencies.

```
pip install -r requirements.txt
```

4. Configure .env file with your settings.
5. Add your PDF/TXT/DOCX files in data/documents/
6. Run the project.

```
python main.py
```

7. Ask questions and get answers from your documents.

7. Expected Output

```
$ python main.py
Loading documents...
Loaded 3 documents
Splitting documents into chunks...
Created 120 chunks
Generating embeddings...
Storing embeddings in Qdrant...
✅ Ingestion completed!

RAG Pipeline initialized successfully!
Ask your question (or type 'exit'):
> What is RAG ?
Answer:
RAG (Retrieval Augmented Generation) is a technique
that combines document retrieval with LLM generation
to provide accurate and context-aware answers.
```

Rahul Yadav

app/ingestion.py – Load and Read Documents

1. Purpose

- Load documents from data/documents/ folder
- Support multiple file types (PDF, TXT, DOCX)
- Use LangChain document loaders
- Return a list of Document objects with content and metadata

2. Supported File Types

File Type	Loader (LangChain)	Description
PDF	PyPDFLoader	Reads PDF files page by page
TXT	TextLoader	Reads plain text files
DOCX	Docx2txtLoader	Reads Word (.docx) files

3. app/ingestion.py (Complete Code)

```
# app/ingestion.py
import os
from typing import List
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders import TextLoader
from langchain_community.document_loaders import Docx2txtLoader
from langchain.schema import Document

def load_documents(data_path: str = "data/documents") -> List[Document]:
    """
    Load all PDF, TXT and DOCX files from the given folder.
    Returns a list of LangChain Document objects.
    """
    documents = []

    if not os.path.exists(data_path):
        print(f"Folder not found: {data_path}")
        return documents

    for file in os.listdir(data_path):
        file_path = os.path.join(data_path, file)
        if not os.path.isfile(file_path):
            continue
        file_extension = file.lower().split(".")[-1]

        try:
            if file_extension == "pdf":
                loader = PyPDFLoader(file_path)
            elif file_extension == "txt":
                loader = TextLoader(file_path, encoding='utf-8')
            elif file_extension == "docx":
                loader = Docx2txtLoader(file_path)
            else:
                print(f"Skipping unsupported file: {file}")
                continue

            docs = loader.load()

            for doc in docs:
                doc.metadata["source"] = file
                doc.metadata["file_path"] = file_path
                doc.metadata["file_type"] = file_extension
            documents.extend(docs)

            print(f"Loaded {file} -> {len(docs)} documents")

        except Exception as e:
            print(f"Error loading {file}: {e}")

    return documents
```

4. Code Explanation (Line by Line)

- Import required LangChain document loaders.
- load_documents() function scans the data/documents/ folder.
- For each file, it checks the file extension.
- Uses appropriate loader based on file type:
  - PDF → PyPDFLoader
  - TXT → TextLoader
  - DOCX → Docx2txtLoader
- Loads the file and returns a list of Document objects.
- Adds metadata (source, file_path, file_type) for each document.
- Skips unsupported file types.
- Handles errors using try-except.
- Returns a combined list of all loaded documents.

5. Sample Output (Console)

```
Loaded sample.pdf -> 12 documents
Loaded notes.txt -> 1 documents
Loaded guide.docx -> 5 documents

Total documents loaded: 18
```

6. Document Object (Example)

```
Document(
    page_content='This is a sample content from PDF...',
    metadata={
        'source': 'sample.pdf',
        'file_path': 'data/documents/sample.pdf',
        'file_type': 'pdf',
        'page': 0
    }
)
```

7. Folder Example (data/documents/)

```
data/
 ├── documents/
 │    ├── sample.pdf
 │    ├── notes.txt
 │    ├── guide.docx
 │    └── ... (more files)
```

All these files will be loaded by ingestion.py

8. Usage Example (in main.py)

```
from app.ingestion import load_documents

docs = load_documents("data/documents")

print(f"Total documents: {len(docs)}")

if docs:
    print("First document content:")
    print(docs[0].page_content[:200])
```

9. Important Notes

- Make sure data/documents/ folder exists.
- Supported formats: PDF, TXT, DOCX.
- You can extend this to support more formats (CSV, MD, HTML, etc.).
- The returned documents will be used in chunking.py.

Rahul Yadav

app/chunking.py – Split Text into Chunks

1. Purpose

- Split loaded document text into smaller chunks.
- Helps in better embedding and retrieval.
- Uses RecursiveCharacterTextSplitter from LangChain.
- Maintains chunk overlap for context continuity.
- Returns a list of Document objects (chunks) with metadata.

2. Why Chunking is Important?

- LLMs and embedding models have a token limit.
- Large documents are split into smaller pieces.
- Improves retrieval accuracy.
- Overlap helps maintain context between chunks.
- Each chunk becomes a searchable unit in vector store.

3. app/chunking.py (Complete Code)

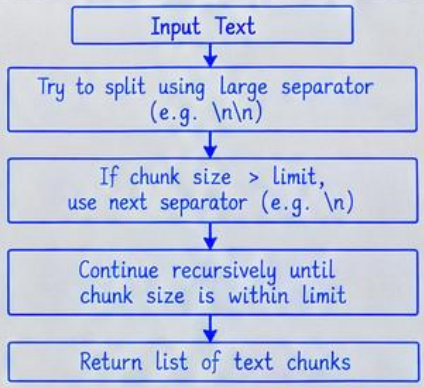
```python
# app/chunking.py
from typing import List
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.schema import Document

def split_documents(
    documents: List[Document],
    chunk_size: int = 500,
    chunk_overlap: int = 50,
    separators: List[str] = None
) -> List[Document]:
    """
    Split documents into smaller chunks using
    RecursiveCharacterTextSplitter.

    Args:
        documents: List of LangChain Document objects
        chunk_size: Size of each chunk (in characters)
        chunk_overlap: Overlap between chunks (in characters)
        separators: Custom separators for splitting
    Returns:
        List of chunked Document objects
    """
    if separators is None:
        separators = ["\n\n", "\n", ".", ".", "!", "?", ","]
    text_splitter =  RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=separators,
    )
    chunks = text_splitter.split_documents(documents)
    # Add chunk metadata
    for i, chunk in enumerate(chunks):
        chunk.metadata["chunk_id"]  = i
        chunk.metadata["chunk_size"] = len(chunk.page_content)
        chunk.metadata["source"] = chunk.metadata.get("source","")

    return chunks
```

4. How RecursiveCharacterTextSplitter Works?



5. Parameters Explanation

Parameter	Description
chunk_size	Maximum size of each chunk (in characters)
chunk_overlap	Number of overlapping characters between consecutive chunks
length_function	Function to calculate length (default: len)
separators	List of separators used for splitting (e.g, paragraph, line break, space)

6. Example: Chunking a Long Text

Input Text (simplified):

> LangChain is a framework for building LLM applications. It provides tools for document loading, text splitting, embeddings and vector stores. It helps in building RAG applications easily.

After Chunking (chunk_size=50, chunk_overlap=10):

Chunk 1:
LangChain is a framework for building LLM applications. It

Chunk 2:
applications. It provides tools for document loading, text

Chunk 3:
loading, text splitting, embeddings and vector stores. It helps in ...

7. Usage Example (in main.py)

```python
from app.ingestion import load_documents
from app.chunking import split_documents

# Load documents
docs = load_documents("data/documents")
print(f"Loaded {len(docs)} documents")

# Split into chunks
chunks = split_documents(docs, chunk_size=500, chunk_overlap=50)
print(f"Created {len(chunks)} chunks")

# Check first chunk
print("First chunk content:")
print(chunks[0].page_content[:200])
print("Metadata:", chunks[0].metadata)
```

8. Sample Output (Console)

```
Loaded 3 documents
Created 120 chunks

First chunk content:
LangChain is a framework for building LLM applications.
It provides tools for document loading, text splitting,
embeddings and vector stores...

Metadata:
{  'source': 'data/documents/sample.pdf',
   'file_type': 'pdf',
   'chunk_id': 0,
   'chunk_size': 482,
    'page': 0
}
```

9. Important Notes

- Adjust chunk_size and chunk_overlap based on your data and use case.
- Smaller chunks improve retrieval but increase storage size.
- Overlap helps maintain context between chunks.
- You can use custom separators for better splitting (e.g. headings, paragraphs).
- The returned chunks are used for embeddings in embeddings.py.

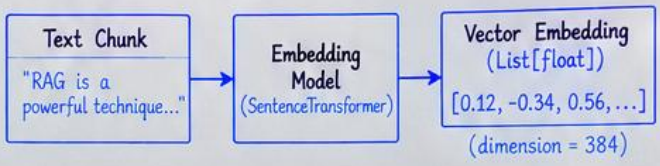
Rahul Yadav

app/embeddings.py – Create Embeddings

1. Purpose

- Convert text chunks into vector embeddings.
- Uses HuggingFace SentenceTransformer model.
- Returns embeddings for each chunk.
- These embeddings will be stored in Qdrant vector database.
- Provides a reusable function to get embeddings.

2. How Embeddings Work ?



3. app/embeddings.py (Complete Code)

```python
# app/embeddings.py
from typing import List
from langchain_huggingface import HuggingFaceEmbeddings
from config.settings import settings

class EmbeddingService:
    """
    Creates text embeddings using HuggingFace model.
    """
    def __init__(self):
        # Initialize HuggingFace embedding model
        self.model_name = settings.EMBEDDING_MODEL
        self.model_kwargs = {"device": "cpu"}  # use "cuda" if GPU available
        self.encode_kwargs = {"normalize_embeddings": True}

        self.embeddings = HuggingFaceEmbeddings(
            model_name=self.model_name,
            model_kwargs=self.model_kwargs,
            encode_kwargs=self.encode_kwargs
        )

    def get_embeddings(self, texts: List[str]) -> List[List[float]]:
        """
        Converts a list of texts into embeddings.
        Args:
            texts: List of text chunks
        Returns:
            List of embeddings (each embedding is a list of floats)
        """
        if not texts:
            return []
        embeddings = self.embeddings.embed_documents(texts)
        return embeddings

    def get_query_embedding(self, query: str) -> List[float]:
        """
        Converts a single query into embedding.
        """
        return self.embeddings.embed_query(query)
```

4. Explanation (Line by Line)

- Import HuggingFaceEmbeddings from LangChain.
- Load model name from config/settings.py.
- model_kwargs specifics device (CPU or GPU).
- encode_kwargs is used to normalize embeddings (normalization helps in better similarity search).
- get_embeddings() takes a list of texts and returns a list of embedding vectors.
- get_query_embedding() converts a single query into an embedding vector.
- Uses SentenceTransformer model (default: all-MiniLM-L6-v2) which gives 384 dimensional embeddings.

5. Embedding Model Details

Model Name	all-MiniLM-L6-v2
Model Type	SentenceTransformer (HuggingFace)
Embedding Dimension	384
Max Input Length	256 tokens (automatically handled)
Advantages	• Fast and lightweight • Good quality embeddings • Open source (free) • Works well for semantic search
Use Case	Text search, RAG, similarity matching

6. Example Usage

```python
from app.embeddings import EmbeddingService
service = EmbeddingService()

texts = [
    "RAG is a technique for building LLM applications.",
    "It combines retrieval and generation.",
    "Qdrant is a vector database.",
]

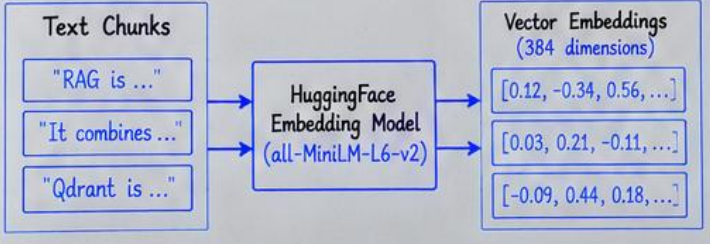
embeddings = service.get_embeddings(texts)
print(f"Generated {len(embeddings)} embeddings")
print(f"Embedding dimension: {len(embeddings[0])}")

# Query embedding
query = "What is RAG?"
query_embedding = service.get_query_embedding(query)
print(f"Query embedding dimension: {len(query_embedding)}")
```

7. Sample Output (Console)

```
Generated 3 embeddings
Embedding dimension: 384
Query embedding dimension: 384
First embedding (first 5 values):
[0.0123, -0.0345, 0.0567, 0.0891, -0.0234, ...]
```

8. Visual Representation



9. Important Notes

- We are using all-MiniLM-L6-v2 which is fast and efficient.
- You can also use other models like:
  - sentence-transformers/all-Mpnet-base-v2 (768 dims)
  - BAAI/bge-small-en (384 dims)
- Use GPU (cuda) for faster embedding generation if available.
- Keep the same embedding model for indexing and querying.
- Embeddings are used in vector_store.py to store in Qdrant.

Rahul Yadav

app/vector_store.py - Store in Qdrant

1. Purpose

- Store document embeddings in Qdrant vector database.
- Create collection if it does not exist.
- Add text chunks with metadata.
- Perform similarity search to retrieve relevant chunks.
- Return top-k similar documents for a given query.

2. What is Qdrant ?

- Qdrant is an open-source vector database.
- Stores vector embeddings with metadata (payload).
- Supports fast similarity search.
- Can be used locally or in cloud.
- Provides filter, payload search and high performance.
- Perfect for RAG applications.

3. app/vector_store.py (Complete Code)

```
# app/vector_store.py
from typing import List, Dict, Any
from qdrant_client import QdrantClient
from qdrant_client.http.models import (
    VectorParams,
    Distance,
    PointStruct,
    Filter,
    FieldCondition,
    MatchValue
)
from config.settings import settings
from app.embeddings import EmbeddingService

class VectorStoreService:
    """
    Handles Qdrant vector store operations.
    """
    def __init__(self):
        self.client = QdrantClient(url=settings.QDRANT_URL)
        self.collection_name = settings.QDRANT_COLLECTION
        self.embedding_service = EmbeddingService()
        self._create_collection()

  def _create_collection(self):
      """Create collection if it does not exist."""
      collections = self.client.get_collections()
      collection_names = [c.name for c in collections.collections]
      if self.collection_name not in collection_names:
          print(f"Creating collection: {self.collection_name}")
          self.client.create_collection(
              collection_name=self.collection_name,
              vectocs_config=VectorParams(
                  size=settings.EMBEDDING_DIM,
                  distance=Distance.COSINE
              )
          )
      else:
          print(f"Collection already exists: {self.collection_name}")
```

4. Adding Documents to Qdrant

```
def add_documents(self, chunks: List[Dict[str, Any]]):
    """
    Add text chunks to Qdrant.
    Each chunk is a dict: {"content": str, "metadata": dict}
    """
    texts = [chunk["content"] for chunk in chunks]
    embeddings = self.embedding_service.get_embeddings(texts)

    points = []
    for idx, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
        point = PointStruct(
            id=idx,
            vector=embedding,
            payload=chunk["metadata"]
        )
        points.append(point)

    self.client.upsert(
        collection_name=self.collection_name,
        points=points
    )
    print(f"Added {len(points)} documents to Qdrant.")
```

5. Similarity Search

```
def similarity_search(self, query: str, k: int = 4,
    filter_metadata: dict = None) -> List[Dict[str, Any]]:
    """
    Search similar chunks from Qdrant.
    """
    query_embedding = self.embedding_service.get_query_embedding(query)

    query_filter = None
    if filter_metadata:
        conditions = []
        for key, value in filter_metadata.items():
            conditions.append(
              FieldCondition(key=key, match=MatchValue(value=value))
            )
        query_filter = Filter(must=conditions)

    results = self.client.search(
        collection_name=self.collection_name,
        query_vector=query_embedding,
        limit=k,
        query_filter=query_filter
    )

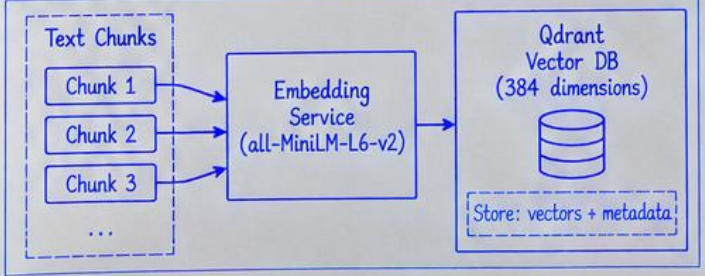
    return [
        {"content": hit.payload.get("content", ""),
         "metadata": hit.payload,
         "score": hit.score}
    ]   for hit in results
```

6. Searching with Metadata Filter (Example)

```
# Search only from specific file type
results = vector_store.similarity_search(
    query="What is RAG?",
    k=3,
    filter_metadata={"file_type": "pdf"}
)

for r in results:
    print("Score:", r["score"])
    print("Content:", r["content"][:200])
    print("Metadata:", r["metadata"])
```

7. Flow Diagram



8. Sample Output (Console)

```
Creating collection: rag_docs
Added 120 documents to Qdrant.
Search results for: "What is RAG?"
------------------------------------------------

1. Score: 0.8123
   Content: "RAG (Retrieval Augmented Generation) ..."
   Metadata: {'source': 'sample.pdf', 'page': 2, 'file_type': 'pdf'}

2. Score: 0.7956
   Content: "It combines retrieval and generation ..."
   Metadata: {'source': 'guide.docx', 'page': 1, 'file_type': 'docx'}
```

9. Important Notes

- Use cosine distance for text embeddings.
- Store useful metadata like source, page, file_type etc.
- You can delete collection using client.delete_collection().
- Qdrant supports filter search to restrict results.
- Use persistent storage in production.

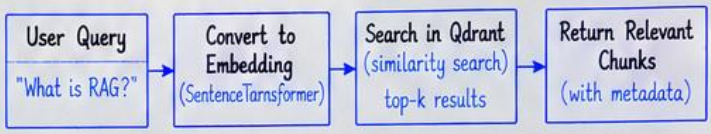
Rahul Yadav

app/retriever.py – Retrieve Relevant Chunks

1. Purpose

- Search relevant document chunks from Qdrant.
- Use embedding similarity to find top-k results.
- Support metadata filters (file type, source, etc).
- Return the most relevant chunks with metadata.
- This module is used to fetch context for LLM.

2. How Retrieval Works ?



3. app/retriever.py (Complete Code)

```python
# app/retriever.py
from typing import List, Dict, Any, Optional
from qdrant_client import QdrantClient
from qdrant_client.http.models import (
    Filter, FieldCondition, MatchValue, MetadataFilter,
    SearchRequest
)
from config.settings import settings
from app.embeddings import EmbeddingService

class Retriever:
    """
    Handles retrieval of relevant document chunks from Qdrant.
    """
    def __init__(self):
        self.client = QdrantClient(url=settings.QDRANT_URL)
        self.collection_name = settings.QDRANT_COLLECTION
        self.embedding_service = EmbeddingService()

    def retrieve(self, query: str, k: int = 4,
                 filter_metadata: Optional[Dict[str, Any]] = None) -> List[Dict[str, Any]]:
        """
        Retrieve top-k relevant chunks for the given query.
        """
        # 1. Convert query to embedding
        query_embedding = self.embedding_service.get_query_embedding(query)

        # 2. Build filter (if provided)
        query_filter = None
        if filter_metadata:
            conditions = []
            for key, value in filter_metadata.items():
                conditions.append(
                    FieldCondition(key=key, match=MatchValue(value=value))
                )
            query_filter = Filter(must=conditions)

        # 3. Search in Qdrant
        search_results = self.client.search(
            collection_name=self.collection_name,
            query_vector=query_embedding,
            limit=k,
            query_filter=query_filter
        )

        # 4. Format results
        results = []
        for hit in search_results:
            results.append({
                "content": hit.payload.get("content", ""),
                "metadata": {
                    "source": hit.payload.get("source", ""),
                    "file_type": hit.payload.get("file_type", ""),
                    "page": hit.payload.get("page", -1),
                    "chunk_id": hit.payload.get("chunk_id", -1),
                    "score": float(hit.score)
                }
            })
        return results
```

4. Explanation (Line by Line)

- Import QdrantClient and required models for search and filtering.
- Use EmbeddingService to convert the query to embedding.
- retrieve() function takes query, top-k value and optional filters.
- If filter_metadata is provided, we create FieldCondition for each key.
- client.search() performs similarity search in Qdrant.
- Returns a list of relevant chunks with their content and metadata.
- Each result contains source, file_type, page, chunk_id and similarity score.

5. Parameters Explanation

Parameter	Description
query	User question or search query (string)
k	Number of top results to retrieve (default: 4)
filter_metadata	Optional metadata filters (e.g., {"file_type": "pdf"})
query_embedding	Vector embedding of the query
limit	Number of results to return from Qdrant
query_filter	Filter object to restrict search (optional)

6. Metadata Filter Example

```python
# Retrieve only PDF documents
results = retriever.retrieve(
    query="What is RAG?",
    k=4,
    filter_metadata={"file_type": "pdf"}
)

# Retrieve only from a specific source
results = retriever.retrieve(
    query="Explain Qdrant.",
    k=4,
    filter_metadata={"source": "guide.pdf"}
)
```

7. Sample Output (Console)

```
Query: What is RAG?
Top 3 results:
1. Score: 0.8123
   Content: RAG (Retrieval Augmented Generation) is ...
   Metadata: {'source': 'sample.pdf', 'file_type': 'pdf', 'page': 2}

2. Score: 0.7651
   Content: It combines retrieval and generation ...
   Metadata: {'source': 'guide.docx', 'file_type': 'docx', 'page': 1}

3. Score: 0.7023
   Content: Qdrant is a vector database ...
   Metadata: {'source': 'notes.txt', 'file_type': 'txt', 'page': 0}
```

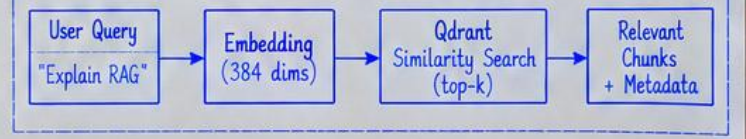
8. Usage Example (in main.py)

```python
from app.retriever import Retriever

retriever = Retriever()
query = "What is RAG?"
results = retriever.retrieve(query, k=4)

for i, r in enumerate(results):
    print(f"\nResult {i+1} (score: {r['metadata']['score']}):")
    print(r["content"][:200])
    print("Metadata:", r["metadata"])
```

9. Flow Diagram



10. Important Notes

- Use appropriate k value based on your use case.
- Metadata filters help in domain-specific retrieval.
- Higher k gives more context but increases token usage.
- You can apply filters like file_type, source, page, etc.
- This retriever returns chunks which will be passed to LLM in rag_pipeline.py.

Rahul Yadav

app/llm.py – LLM Integration (Ollama / OpenAI)

1. Purpose

- Integrate LLM (Ollama or OpenAI) for RAG.
- Provide a common interface to load and use LLM.
- Use prompt template for better responses.
- Return final answer based on retrieved context.
- Support both local (Ollama) and cloud (OpenAI) models.

2. Supported LLM Models

Model Type	LangChain Class	Example Model Name	Use Case
Ollama (Local)	ChatOllama	llama3 mistral phi3	Free, local, private
OpenAI (Cloud)	ChatOpenAI	gpt-3.5-turbo gpt-4o	High quality, production use

3. app/llm.py (Complete Code)

```python
# app/llm.py
from typing import List
from langchain_openai import ChatOpenAI
from langchain_community.chat_models import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import HumanMessage, SystemMessage
from config.settings import settings

class LLMService:
    """
    Handles LLM integration using OpenAI or Ollama.
    """

    def __init__(self):
        # Choose model based on settings
        if settings.LLM_MODEL.lower() == "ollama":
            self.llm = ChatOllama(
                model=settings.OLLAMA_MODEL,
                base_url=settings.OLLAMA_BASE_URL,
                temperature=settings.TEMPERATURE
            )
        else:
            self.llm = ChatOpenAI(
                model=settings.OPENAI_MODEL,
                openai_api_key=settings.OPENAI_API_KEY,
                base_url=settings.OPENAI_BASE_URL,
                temperature=settings.TEMPERATURE
            )
        # Prompt template for RAG
        self.prompt = ChatPromptTemplate.from_messages([
            ("system", "You are a helpful AI assistant. \
            Answer the question using the provided context. \
            If the answer is not in the context, say you don't know."),
            ("human", "Context:\n{context}\n\nQuestion: {question}")
        ])

    def get_answer(self, question: str, context, str) -> str:
        """
        Generate answer using LLM with given context.
        """
        chain = self.prompt | self.llm
        response = chain.invoke({
            "context": context,
            "question": question
        })
        return response.content
```

4. Explanation (Line by Line)

- Import necessary LangChain modules for OpenAI and Ollama.
- Read model configuration from settings.py.
- If LLM_MODEL is "ollama", initialize ChatOllama with model name and base URL.
- If LLM_MODEL is "openai", initialize ChatOpenAI with API key.
- Create a prompt template with system and human messages.
- get_answer() function takes question and retrieved context and returns the LLM response.
- The prompt ensures the model uses only the provided context and avoids hallucination.

5. Prompt Template Used

System Prompt:
You are a helpful AI assistant. Answer the question using the provided context. If the answer is not in the context, say you don't know.

Human Prompt:

```
Context:
{context}
\
Question: {question}
```

6. Example Usage (in main.py or rag_pipeline.py)

```python
from app.llm import LLMService

llm_service = LLMService()

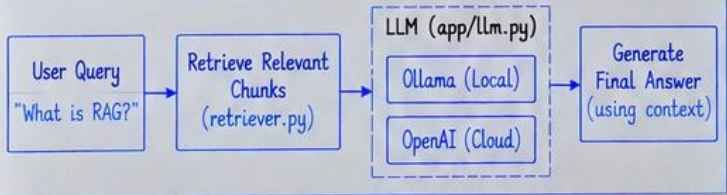
context = """
    RAG (Retrieval Augmented Generation) combines retrieval
    and generation. It uses a vector database to retrieve
    relevant documents and an LLM to generate answers.
    """
question = "What is RAG?"
answer = llm_service.get_answer(question, context)
print("Answer:", answer)
```

7. Sample Output (Console)

```
Question: What is RAG?

Answer:
RAG (Retrieval Augmented Generation) combines information
retrieval with text generation. It retrieves relevant
documents from a vector database and uses an LLM to
generate accurate and context-aware answers.
```

8. Integration Flow



9. Important Notes

- Set LLM_MODEL in .env file (ollama or openai).
- For OpenAI, provide valid OPENAI_API_KEY.
- For Ollama, make sure Ollama is running locally (http://localhost:11434).
- Adjust temperature based on use case (0.0 for factual, 0.7 for creative).
- Use a clear and concise prompt to get better results.
- This module is used in rag_pipeline.py to generate final answer.

10. Environment Variables (.env)

```
# LLM Configuration
LLM_MODEL=ollama  # or openai

# For Ollama (Local)
OLLAMA_MODEL=llama3
OLLAMA_BASE_URL=http://localhost:11434

# For OpenAI (Cloud)
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-3.5-turbo
OPENAI_BASE_URL=

# Common
TEMPERATURE=0.2
```

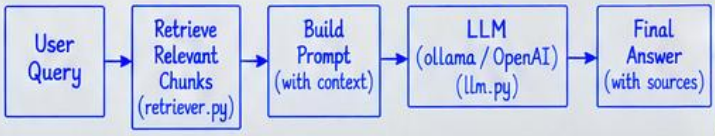
Rahul Yadav

app/rag_pipeline.py – Complete RAG Pipeline

1. Purpose

- Orchestrate the complete RAG flow.
- Take user query as input.
- Retrieve relevant document chunks from Qdrant.
- Pass retrieved context to LLM (Ollama/OpenAI).
- Return final answer with source documents.
- This module combines ingestion, retrieval and LLM.

2. RAG Pipeline Flow



3. app/rag_pipeline.py (Complete Code)

```python
# app/rag_pipeline.py
from typing import List, Dict, Any
from config.settings import settings
from app.retriever import Retriever
from app.llm import LLMService
from app.ingestion import load_documents
from app.chunking import split_documents
from app.embeddings import EmbeddingService
from app.vector_store import VectorStoreService

class RAGPipeline:
    """
    Complete RAG pipeline to ingest documents,
    store embeddings, retrieve and generate answer.
    """

    def __init__(self):
        # Initialize all services
        self.embedding_service = EmbeddingService()
        self.vector_store = VectorStoreService()
        self.retriever = Retriever()
        self.llm_service = LLMService()

    def ingest_and_index(self, data_path: str):
        """
        Load documents, split into chunks, create embeddings
        and store in Qdrant.
        """
        print("Step 1: Loading documents...")
        documents = load_documents(data_path)
        print(f"Loaded {len(documents)} documents.")

        print("Step 2: Splitting documents into chunks...")
        chunks = split_documents(documents,
                 chunk_size=settings.CHUNK_SIZE,
                 chunk_overlap=settings.CHUNK_OVERLAP)
        print(f"Created {len(chunks)} chunks.")

        print("Step 3: Creating embeddings...")
        embeddings = self.embedding_service.get_embeddings([
            chunk.page_content for chunk in chunks
        ])

        print("Step 4: Storing in Qdrant...")
        self.vector_store.add_documents(chunks, embeddings)
        print("Ingestion and indexing completed successfully.")
```

4. Ask Question (RAG)

```python
def ask(self, query: str, k: int = 4, filter_metadata: Dict[str, Any] = None
    ) -> Dict[str, Any]:
    """
    Retrieve relevant chunks and generate answer using LLM.
    Returns answer and source documents.
    """
    # 1. Retrieve relevant chunks
    print(f"Retrieving top {k} chunks for query: {query}")
    results = self.retriever.retrieve(
        query=query,
        k=k,
        filter_metadata=filter_metadata
    )
    if not results:
        return {
            "answer":"No relevant documents found.",
            "sources": []
        }

    # 2. Prepare context from retrieved chunks
    context = "\n\n".join([r["content"] for r in results])

    # 3. Generate answer using LLM
    answer = self.llm_service.get_answer(query, context)

    # 4. Prepare source information
    sources = []
    for r in results:
        sources.append({
            "content": r["content"],
            "metadata": r["metadata"],
            "score": r["score"]
        })

    return {
        "answer": answer,
        "sources": sources
    }
```

5. Example Usage (in main.py)

```python
from app.rag_pipeline import RAGPipeline

pipeline = RAGPipeline()

# First time: ingest and index documents
pipeline.ingest_and_index("data/documents")

# Ask a question
query = "What is RAG?"
result = pipeline.ask(query, k=3)
print("Answer:\n", result["answer"])
print("\nSources:")
for i, src in enumerate(result["sources"]):
    print(f"\nSource {i+1}:")
    print("Score:", src["score"])
    print("Metadata:", src["metadata"])
    print("Content Preview:", src["content"][:200])
```

6. Sample Output (Console)

```
Step 1: Loading documents...
Loaded 3 documents.
Step 2: Splitting documents into chunks...
Created 120 chunks.
Step 3: Creating embeddings...
Step 4: Storing in Qdrant...
Ingestion and indexing completed successfully.

Retrieving top 3 chunks for query: What is RAG?

Answer:
RAG (Retrieval Augmented Generation) is a technique that combines
information retrieval with text generation. It retrieves relevant
documents from a vector database and uses an LLM to generate
accurate and context-aware answers.

Sources:
Source 1:
Score: 0.8123
Metadata: {'source': 'rag_guide.pdf', 'page': 2, 'file_type': 'pdf'}
Content Preview: RAG is a powerful technique for building LLM...
```

7. Important Notes

- Run ingest_and_index() only once (or when adding new documents).
- You can adjust k based on your use case.
- Use metadata filters to restrict search to specific files/folders.
- The context passed to LLM should not exceed token limit.
- You can improve the prompt for better answers.
- This pipeline can be extended with caching, streaming, or chat history.

8. Next Steps

- Use this pipeline in a Streamlit app (main.py).
- Add chat history support.
- Implement source citation in the final answer.
- Add advanced features like reranking, hybrid search, or multi-query.

Rahul Yadav

main.py – Entry Point & Complete RAG Application

1. Purpose

- This is the main entry point of the RAG application.
- Initialize all services (ingestion, embedding, vector store, retriever, LLM).
- Index documents from the data folder.
- Provide a simple command line interface to ask questions.
- This file ties together the complete RAG pipeline.

2. Project Flow



3. main.py (Complete Code)

```python
# main.py
import os
from config.settings import settings
from app.rag_pipeline import RAGPipeline

def print_banner():
    print("=" * 60)
    print("     RAG DOCUMENT ASSISTANT")
    print("=" * 60)
    print("Type 'exit' to quit.\n")

def index_documents(pipeline: RAGPipeline, data_path: str):
    """Load, process and index documents."""
    print(f"📁 Indexing documents from: {data_path}")
    pipeline.ingest_and_index(data_path)
    print("✅ Documents indexed successfully!\n")

def ask_question(pipeline: RAGPipeline):
    """Continuously take user questions and return answers."""
    while True:
        query = input("🔍 Ask a question: ").strip()
        if query.lower() in ["exit", "quit"]:
            print("👋 Goodbye!")
            break
        if not query:
            print("Please enter a valid question.\n")
            continue

        print("\n⏳ Generating answer...\n")
        result = pipeline.ask(query, k=4)

        answer = result.get("answer", "")
        sources = result.get("sources", [])

        print("💡 Answer:\n")
        print(answer)

        print("\n📄 Sources:")
        if not sources:
            print("No sources found.")
        else:
            for i, src in enumerate(sources, 1):
                content = src.get("content", "")[:200].replace("\n", " ")
                metadata = src.get("metadata", {})
                score = src.get("score", 0.0)
                print(f"\n[{i}] Score: {score}")
                print(f"Source: {metadata.get('source', 'N/A')}")
                print(f"Page: {metadata.get('page', 'N/A')}")
                print(f"Content: {content} ...")

def main():
    print_banner()
    # Initialize RAG pipeline
    pipeline = RAGPipeline()
    # Index documents
    index_documents(pipeline, settings.DATA_PATH)
    # Start asking questions
    ask_question(pipeline)
if __name__ == "__main__":
    main()
```

4. Explanation (Line by Line)

- Import settings and RAGPipeline from our modules.
- print_banner() shows application name and instructions.
- index_documents() loads documents from the data folder and indexes them in Qdrant.
- ask_question() runs a loop to take user input, calls pipeline.ask() and prints the answer with sources.
- main() initializes the pipeline, indexes documents and starts the interactive QA loop.
- You can modify data_path, number of results (k), or add additional CLI options as per your use case.

5. Configuration in .env

```
# LLM Configuration (choose one)
LLM_MODEL=ollama  # or openai
OLLAMA_MODEL=llama3
OLLAMA_BASE_URL=http://localhost:11434
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-3.5-turbo
OPENAI_BASE_URL=

# Qdrant Configuration
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=rag_docs
CHUNK_SIZE=500
CHUNK_OVERLAP=50
EMBEDDING_MODEL=all-MiniLM-L6-v2

# App Configuration
DATA_PATH=data/documents
TOP_K=4
TEMPERATURE=0.2
```

6. Running the Application

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Start Qdrant (if not running)
docker run -p 6333:6333 qdrant/qdrant

# 3. Start Ollama (if using local model)
ollama pull llama3

# 4. Run the application
python main.py
```

7. Sample Console Output

```
============================================
          RAG DOCUMENT ASSISTANT
============================================
Type 'exit' to quit.

📁 Indexing documents from: data/documents
✅ Documents indexed successfully!

🔍 Ask a question: What is RAG?

💡 Answer:
RAG (Retrieval Augmented Generation) is a technique ...

📄 Sources:
[1] Score: 0.8123
    Source: rag_guide.pdf
    Page: 2
    Content: RAG combines information retrieval with ...
```

8. Important Notes

- Make sure .env file is properly configured.
- Place your PDF, TXT, DOCX files in data/documents/ folder.
- Run index_documents() again whenever you add new documents.
- You can change TOP_K to get more or fewer results.
- This is a simple CLI version; you can extend it to a Streamlit or FastAPI application.

Rahul Yadav

requirements.txt – Dependencies, Version Management & Installation

1. Purpose

- List all required Python packages for the RAG project.
- Ensure consistent environment across different systems.
- Specify exact or compatible versions of dependencies.
- Make it easy to install the project using a single command.
- Avoid version conflicts and ensure reproducibility.
- Useful for deployment (Docker, Cloud, Server, etc).

3. requirements.txt (Minimal Version)

If you want a minimal setup to just run the basic RAG project:

```
langchain==0.2.11
langchain-community==0.2.11
langchain-ollama==0.1.4
sentence-transformers==2.7.0
qdrant-client==1.9.1
pypdf==4.2.0
python-docx==1.1.2
python-dotenv==1.0.1
tqdm==4.66.4
```

4. Installing Dependencies

1. Create a virtual environment (Recommended)

```
# Windows
python -m venv .venv
.venv\Scripts\activate
```

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
```

2. Install required packages

```
pip install -r requirements.txt
```

3. Verify installation

```
pip list
```

6. Using requirements.txt in Different Scenarios

Scenario	Command	Description
Local Development	pip install -r requirements.txt	Install all dependencies
Docker	COPY requirements.txt . RUN pip install -r requirements.txt	Install inside Docker image
Deployment (Server)	pip install -r requirements.txt	Install on server
Update Packages	pip freeze > requirements.txt	Save current versions
Minimal Setup	Use minimal version file	For basic RAG setup

8. Docker Example (Using requirements.txt)

```
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

2. Complete requirements.txt (Recommended)

```
# Core Framework
langchain==0.2.11                    # LangChain framework
langchain-community==0.2.11          # Community integrations
langchain-core==0.2.27               # Core components

# LLM Integration
langchain-ollama==0.1.4              # Ollama integration (local)
langchain-openai==0.1.22             # OpenAI integration (cloud)
openai==1.40.0                       # OpenAI Python client

# Embeddings
sentence-transformers==2.7.0         # For local embeddings
huggingface-hub==0.24.6              # Model downloads
numpy==1.26.4                        # Numerical operations

# Vector Database
qdrant-client==1.9.1                 # Qdrant vector database

# Document Processing
pypdf==4.2.0                         # PDF loader
python-docx==1.1.2                   # DOCX loader
unstructured==0.14.6                 # Additional file loaders
unstructured[all-docs]==0.14.6       # Support for more file types
beautifulsoup4==4.12.3               # HTML parsing if needed

# Utils
python-dotenv==1.0.1                 # Load environment variables
tqdm==4.66.4                         # Progress bar
pandas==2.2.2                        # Data handling
fastapi==0.111.0                     # (Optional) API version
streamlit==1.35.0                    # (Optional) Web UI
uvicorn==0.30.1                      # (Optional) Run FastAPI
python-multipart==0.0.9              # (For file uploads)
```

5. Upgrading / Managing Versions

- Upgrade a specific package:

```
pip install --upgrade package_name
```

- Upgrade all packages:

```
pip install --upgrade -r requirements.txt
```

- Freeze current environment (generate requirements.txt):

```
pip freeze > requirements.txt
```

- Check installed version:

```
pip show package_name
```

7. Example Project Structure

```
rag-project/
├── data/
│   └── documents/
├── app/
│   ├── ingestion.py
│   ├── chunking.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── llm.py
│   └── rag_pipeline.py
├── main.py
├── requirements.txt     ← All dependencies listed here
├── .env                 ← Environment variables
├── README.md
└── Dockerfile (optional)
```

9. Important Notes

- Use specific versions to avoid compatibility issues.
- Update versions periodically for new features and security fixes.
- Use a virtual environment.
- Keep requirements.txt in the root folder.
- For production, test the installation in a clean environment.
- You can separate dev dependencies (e.g., streamlit, fastapi) if needed.
- Use pip freeze after final testing to lock versions.

Rahul Yadav

.env Configuration – API Keys, Qdrant, Ollama, OpenAI & Other Settings

1. Purpose

- Store sensitive configuration like API keys and URLs.
- Keep environment specific settings separate from code.
- Use different .env files for local, development and production.
- Make the application secure and easy to configure.
- Load these variables in Python using python-dotenv.

2. .env File (Complete Example)

```
# ============================
# RAG Project - Environment Variables
# ============================

# LLM Configuration
LLM_MODEL=ollama                    # ollama or openai
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-3.5-turbo
OPENAI_BASE_URL=

# Qdrant Configuration
QDRANT_URL=http://localhost:6333
QDRANT_API_KEY=
QDRANT_COLLECTION=rag_docs
CHUNK_SIZE=500
CHUNK_OVERLAP=50
EMBEDDING_MODEL=all-MiniLM-L6-v2

# Application Settings
DATA_PATH=data/documents
TOP_K=4
TEMPERATURE=0.2
MAX_TOKENS=1024
APP_HOST=0.0.0.0
APP_PORT=8501
DEBUG=True
LOG_LEVEL=INFO

# Optional Settings
PERSIST_DIRECTORY=./storage
ALLOWED_FILE_TYPES=pdf,txt,docx
MAX_FILE_SIZE_MB=10
```

3. Loading .env in Python

```python
# Using python-dotenv to load environment variables
from dotenv import load_dotenv
import os

# Load .env file (by default looks for .env in current directory)
load_dotenv()

# Access environment variables
llm_model = os.getenv("LLM_MODEL", "ollama")
ollama_base_url = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
ollama_model = os.getenv("OLLAMA_MODEL", "llama3")
openai_api_key = os.getenv("OPENAI_API_KEY")
openai_model = os.getenv("OPENAI_MODEL", "gpt-3.5-turbo")
qdrant_url = os.getenv("QDRANT_URL", "http://localhost:6333")
qdrant_collection = os.getenv("QDRANT_COLLECTION", "rag_docs")
top_k = int(os.getenv("TOP_K", "4"))
temperature = float(os.getenv("TEMPERATURE", "0.2"))
```

4. Important Environment Variables Explanation

Variable	Description	Example Value
LLM_MODEL	LLM to use (ollama or openai)	ollama
OLLAMA_BASE_URL	Ollama server URL	http://localhost:11434
OLLAMA_MODEL	Model name in Ollama	llama3
OPENAI_API_KEY	OpenAI API key	sk-xxxxxxxxxx
OPENAI_MODEL	OpenAI model name	gpt-3.5-turbo
QDRANT_URL	Qdrant server URL	http://localhost:6333
QDRANT_API_KEY	Qdrant API key (optional)	-
QDRANT_COLLECTION	Vector collection name	rag_docs
CHUNK_SIZE	Text chunk size	500
CHUNK_OVERLAP	Chunk overlap size	50
EMBEDDING_MODEL	Sentence transformer model	all-MiniLM-L6-v2
DATA_PATH	Folder containing documents	data/documents
TOP_K	Number of similar chunks	4
TEMPERATURE	LLM response creativity	0.2
MAX_TOKENS	Maximum tokens in response	1024
APP_HOST	Host for Streamlit/FastAPI	0.0.0.0
APP_PORT	Port for application	8501
DEBUG	Enable debug mode	True
LOG_LEVEL	Logging level	INFO

5. Different .env Files (Environments)

a) .env (Local Development)

```
LLM_MODEL=ollama
OLLAMA_BASE_URL=http://localhost:11434
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=rag_docs
DEBUG=True
APP_PORT=8501
```

b) .env.dev (Development Server)

```
LLM_MODEL=openai
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-3.5-turbo
QDRANT_URL=http://dev-server:6333
QDRANT_COLLECTION=rag_docs_dev
DEBUG=True
```

c) .env.prod (Production)

```
LLM_MODEL=openai
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-4o
QDRANT_URL=https://your-qdrant-server.com
QDRANT_COLLECTION=rag_docs_prod
DEBUG=False
LOG_LEVEL=WARNING
```

6. Accessing .env in Different Modules

```python
# app/config/settings.py
from dotenv import load_dotenv
import os

load_dotenv()

class Settings:
    LLM_MODEL = os.getenv("LLM_MODEL", "ollama")
    OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL")
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
    QDRANT_URL = os.getenv("QDRANT_URL")
    QDRANT_COLLECTION = os.getenv("QDRANT_COLLECTION")
    TOP_K = int(os.getenv("TOP_K", "4"))
    TEMPERATURE = float(os.getenv("TEMPERATURE", "0.2"))
```

7. Security Best Practices

- Never commit .env file to Git.
- Add .env to .gitignore.
- Use different API keys for development and production.
- Rotate API keys periodically.
- Use environment specific .env files.
- Restrict file permissions in production.
- Use secrets management tools (e.g., Docker secrets, AWS Secrets Manager) for production.

8. .gitignore Configuration

```
# Environment variables
.env
.env.*
!.env.example
```

9. Example .env.example (Safe to Commit)

```
LLM_MODEL=ollama
OLLAMA_BASE_URL=http://localhost:11434
OPENAI_API_KEY=your_openai_key_here
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=rag_docs
TOP_K=4
TEMPERATURE=0.2
```

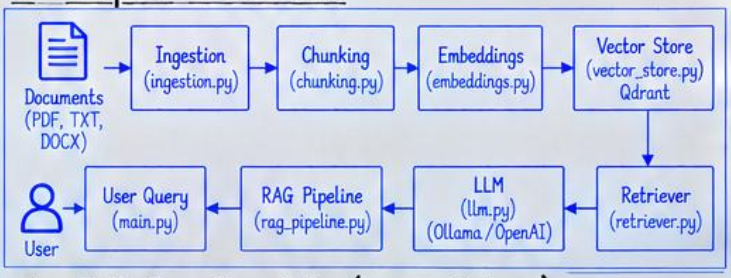
Rahul Yadav

RAG Project Execution – How All Modules Work Together

1. Overview

- This page explains how all modules in our RAG project work together from document ingestion to final answer generation.
- Shows the complete execution flow.
- Explains the role of each module.
- Provides step-by-step process with diagrams.
- Helps to understand the end-to-end working of the project.

2. Complete Architecture



3. Execution Flow (Step by Step)

1. **Load Configuration (.env)**
   - Load API keys, model settings, Qdrant URL, etc.
2. **Initialize Components**
   - Create instances of ingestion, embedding, vector store, retriever and LLM.
3. **Ingest and Index Documents**
   - Load documents from data/ folder.
   - Split into chunks.
   - Generate embeddings.
   - Store in Qdrant vector database.
4. **User Query**
   - Take user input question.
5. **Retrieve Relevant Chunks**
   - Search similar chunks from Qdrant.
6. **Generate Answer using LLM**
   - Build prompt with retrieved context.
   - Call LLM (Ollama or OpenAI).
7. **Return Final Answer**
   - Show answer and source documents.

4. Full Execution Code (run_project.py)

```python
# run_project.py - Complete execution script for RAG project
from dotenv import load_dotenv
from app.ingestion import load_documents
from app.chunking import split_documents
from app.embeddings import EmbeddingService
from app.vector_store import VectorStoreService
from app.retriever import Retriever
from app.llm import LLMService
from app.rag_pipeline import RAGPipeline
from config.settings import settings
import os

def run_rag():
    print("=" * 60)
    print("🚀 Starting RAG Project...")
    print("=" * 60)

    # 1. Load environment variables
    load_dotenv()
    print(f"LLM Model: {settings.LLM_MODEL}")
    print(f"Qdrant URL: {settings.QDRANT_URL}")
    print(f"Data Path: {settings.DATA_PATH}")

    # 2. Initialize pipeline
    pipeline = RAGPipeline()

    # 3. Index documents (run only first time or when new docs added)
    print("\n📂 Indexing documents...")
    pipeline.ingest_and_index(settings.DATA_PATH)

    # 4. Start interactive QA loop
    print("\n✅ Indexing completed!")
    print("\n💬 You can now ask questions (type 'exit' to quit)\n")
    while True:
        query = input("❓ Ask a question: ").strip()
        if query.lower() in ["exit", "quit"]:
            print("👋 Goodbye!")
            braak
        result = pipeline.ask(query, k=4)
        print("\n✨ Answer:")
        print(result.get("answer", "No answer found."))
        print("\n📄 Sources:")
        for i, src in enumerate(result.get("sources", []), 1):
            print(f"[{i}] {src.get('content','')[:200]}...")

if __name__ == "__main__":
    run_rag()
```

5. How Modules Work Together

ingestion.py	Loads documents (PDF, TXT, DOCX, etc.) from data/ folder.
chunking.py	Splits documents into smaller chunks with metadata.
embeddings.py	Converts text chunks into vector embeddings using SentenceTransformer.
vector_store.py	Stores embeddings in Qdrant vector database.
retriever.py	Searches similar chunks from Qdrant using semantic similarity.
llm.py	Generates answer using LLM (Ollama or OpenAI) with given context.
rag_pipeline.py	Combines retrieval and generation with proper prompt template.
main.py	Entry point for the application and handles user interaction.

6. Running the Project

1. Create virtual environment (if not created)

```
python -m venv .venv
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate       # Windows
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Set environment variables in .env (as shown in previous page)
4. Run the application

```
python run_project.py
```

7. Sample Console Output

```
============================================================
🚀 Starting RAG Project...
============================================================
LLM Model: ollama
Qdrant URL: http://localhost:6333
Data Path: data/documents

📂 Indexing documents...
Loaded 5 documents.
Created 42 chunks.
Generated embeddings using all-MiniLM-L6-v2.
Stored 42 vectors in Qdrant.

✅ Indexing completed!

💬 You can now ask questions (type 'exit' to quit)

❓ Ask a question: What is RAG?

✨ Answer:
RAG (Retrieval Augmented Generation) is a technique that combines
information retrieval with text generation. It retrieves relevant
documents from a vector database and uses an LLM to generate
accurate and context-aware answers.

📄 Sources:
[1] rag_guide.pdf - Page 2: RAG is a technique...
[2] ai_basics.pdf - Page 5: Retrieval and Generation...
```

8. Key Points

- Run indexing only once (or when new documents are added).
- You can adjust TOP_K to get more or fewer results.
- Make sure all services (Qdrant, Ollama) are running.
- Check logs if you face any errors.
- This modular structure makes the project easy to maintain and extend.

Rahul Yadav

README.md – Project Documentation, Setup Guide, API Usage & GitHub Repository

1. Purpose

- A good README.md explains your project clearly to others.
- Helps users understand what the project does.
- Provides setup instructions, usage examples and project structure.
- Useful for GitHub repository, portfolio and sharing with others.
- Makes your project professional and easy to use.

2. Recommended README.md Structure

1. Project Title and Description
2. Features
3. Project Architecture (Diagram)
4. Project Structure
5. Prerequisites
6. Installation and Setup
7. Environment Variables (.env)
8. Usage (Indexing Documents and Asking Questions)
9. Example Output
10. Technologies Used
11. Contributing / License

3. Project Structure (for README)

```
rag-project/
├── data/
│   └── documents/   # Put your PDF, TXT, DOCX files here
├── app/
│   ├── ingestion.py
│   ├── chunking.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── llm.py
│   └── rag_pipeline.py
├── main.py          #  Entry point
├── requirements.txt
├── .env             #  Environment variables
├── README.md        #  Project documentation
└── Dockerfile       #  (Optional) Docker setup
```

4. Complete README.md Content (Example)

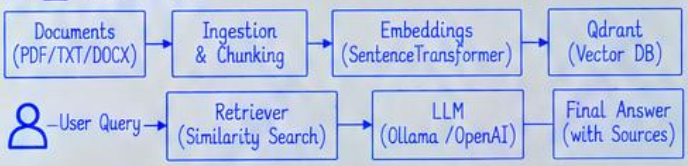
📄 RAG Document Assistant

A simple and powerful Retrieval Augmented Generation (RAG) application that can answer questions from your own documents using LLMs (Ollama or OpenAI) and Qdrant vector database.

✨ Features

- Ingest PDF, TXT, DOCX documents
- Split documents into chunks
- Generate embeddings using Sentence Transformers
- Store embeddings in Qdrant vector database
- Retrieve relevant chunks using semantic search
- Generate accurate answers using LLM (Ollama / OpenAI)
- Show source documents with links and metadata

🏛 Architecture



5. Prerequisites

- Python 3.9+
- Qdrant (Local or Cloud)
- Ollama (for local LLM) or OpenAI API key
- pip package manager

6. Installation and Setup

```
# 1. Clone the repository
```bash
git clone https://github.com/your-username/rag-project.git
cd rag-project
```

2. Create virtual environment
```bash
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate
```

3. Install dependencies
```bash
pip install -r requirements.txt
```

4. Setup environment variables
```bash
cp .env.example .env
# Edit .env file with your API keys and settings
```
```

7. Environment Variables (.env)

```
# Example .env file

# LLM Configuration
LLM_MODEL=ollama            # or openai
OLLAMA_BASE_URL=http://localhost:11434
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-3.5-turbo

# Qdrant Configuration
QDRANT_URL=http://localhost:6333
QDRANT_COLLECTION=rag_docs
CHUNK_SIZE=500
CHUNK_OVERLAP=50
EMBEDDING_MODEL=all-MiniLM-L6-v2

# Application Settings
DATA_PATH=data/documents
TOP_K=4
TEMPERATURE=0.2
```

8. Usage

```
# 1. Put your documents in data/documents/ folder
# 2. Run the application
```bash
python main.py
```

Follow the interactive prompt
Ask a question: What is RAG?
Answer: RAG (Retrieval Augmented Generation) is a technique ...
Sources:
[1] rag_guide.pdf - Page 2
[2] ai_basics.pdf - Page 5
```

9. Example Output

```
$ python main.py
================================
📄 RAG Document Assistant
================================
📂 Indexing documents...
Loaded 3 documents.
Created 28 chunks.
✅ Documents indexed successfully!

Ask a question: What is RAG?

Answer:
RAG (Retrieval Augmented Generation) is a technique that combines
information retrieval with text generation to provide accurate...

Sources:
[1] rag_guide.pdf (Page 2)
[2] ai_basics.pdf (Page 5)

Content Preview:
RAG enables LLMs to use
external knowledge from
your documents...
```

10. Technologies Used

- Python 3.9+
- LangChain / LlamaIndex (optional)
- Sentence Transformers
- Qdrant
- Ollama / OpenAI
- FastAPI / Streamlit (optional)
- PyPDF2, python-docx, unstructured
- Other Python libraries (see requirements.txt)

11. Contributing / License

- Feel free to fork this repository and submit pull requests.
- This project is for educational purposes.
- You can modify and use it for your own projects.
- License: MIT (or any open-source license)

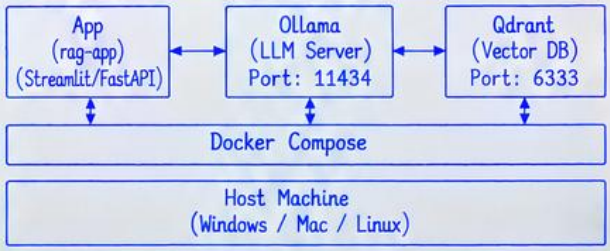
Rahul Yadav

Dockerfile & Docker Compose – Containerize RAG Application

1. Purpose

- Containerize the complete RAG application using Docker.
- Run the application with all dependencies (Qdrant, Ollama, App).
- Ensure consistent environment across different systems.
- Easy deployment and sharing.
- Use Docker Compose to manage multiple services.
- Run everything with a single command.

2. Architecture (Docker Compose)



3. Dockerfile (for RAG Application)

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Install system dependencies (if needed)
RUN apt-get update && apt-get install -y \
      build-essential \
      curl \
      && rm -rf /var/lib/apt/lists/*

# Copy requirements file
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port for Streamlit or FastAPI
# (change port based on your app)
EXPOSE 8501

# Command to run the application (Streamlit example)
CMD ["streamlit", "run", "main.py", "--server.port=8501",
     "--server.address=0.0.0.0"]
```

4. docker-compose.yml (Complete)

```
version: '3.8'
services:
  app:
    build: .
    container_name: rag-app
    ports:
      - "8501:8501"  # Streamlit (change if using FastAPI)
    volumes:
      - ./data:/app/data  # Mount documents folder
    env_file:
      - .env
    depends_on:
      - qdrant
      - ollama
    restart: unless-stopped
    networks:
      - rag-network

  qdrant:
    image: qdrant/qdrant:latest
    container_name: qdrant
    ports:
      - "6333:6333"
    volumes:
      - qdrant_data:/qdrant/storage
    restart: unless-stopped
    networks:
      - rag-network

  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    restart: unless-stopped
    networks:
      - rag-network

volumes:
  qdrant_data:
  ollama_data:
networks:
  rag-network:
    driver: bridge
```

5. .dockerignore (Exclude Unnecessary Files)

```
# Python cache
__pycache__/
*.pyc
*.pyo

# Virtual environment
.venv/
env/

# Git files
.git/
.gitignore

# Data files (if large)
data/
*.pdf
*.txt
*.docx

# Docker files
Dockerfile
docker-compose.yml
```

6. Build and Run Commands

1. Build and start all services
   ```
   docker-compose up --build
   ```
2. Run in background (detached mode)
   ```
   docker-compose up --build -d
   ```
3. Stop all services
   ```
   docker-compose down
   ```
4. View running containers
   ```
   docker ps
   ```
5. View logs
   ```
   docker-compose logs -f
   ```

7. Access the Application

- RAG Application (Streamlit): http://localhost:8501
- Ollama (LLM API): http://localhost:11434
- Qdrant (Dashboard/API): http://localhost:6333/dashboard (or use API)

8. Testing Inside Containers

- Check Ollama models: `docker exec -it ollama ollama list`
- Pull a model (if not available): `docker exec -it ollama ollama pull llama3`
- Check Qdrant collections: `curl http://localhost:6333/collections`
- Access app container shell: `docker exec -it rag-app bash`

9. Sample Output (docker-compose up)

```
$ docker-compose up --build
[+] Building 12.3s (10/10) FINISHED
[+] Running 3/3
 ✓ Container qdrant    Started
 ✓ Container ollama    Started
 ✓ Container rag-app   Started
rag-app  |  You can now view your Streamlit app in your browser.
rag-app  |  Local URL: http://0.0.0.0:8501
rag-app  |  Network URL: http://172.18.0.3:8501
```

10. Important Notes

- Make sure to add your .env file (not committed to Git).
- You can use FastAPI instead of Streamlit (just change the CMD and port).
- Data folder is mounted to persist your documents.
- Ollama will download models automatically or you can pull manually.
- Use volumes to persist Qdrant and Ollama data.
- For production, use proper security, authentication and reverse proxy (e.g., Nginx).

Rahul Yadav

Streamlit UI – Build a Complete RAG Chatbot with Document Upload, Question Input & Source Citations

1. Purpose

- Create a user friendly web interface using Streamlit.
- Allow users to upload documents (PDF, TXT, DOCX).
- Index the uploaded documents in Qdrant vector database.
- Enable question answering using RAG (Ollama or OpenAI).
- Show relevant source documents with citations.
- Maintain chat history and easy to use UI.

2. App Structure (Streamlit UI)

```
rag-project/
├── app/
│   ├── streamlit_app.py   # Main Streamlit UI
│   ├── rag_pipeline.py    # RAG pipeline (from previous pages)
├── data/
│   ├── documents/         # Uploaded documents
├── main.py
├── requirements.txt
├── .env
├── README.md
├── Dockerfile
├── docker-compose.yml
```

3. Streamlit UI Code (streamlit_app.py)

```
import streamlit as st
import os
import tempfile
from patblih import Path
from app.ingestion import load_documents
from app.chunking import split_documents
from app.embeddings import EmbeddingService
from app.vector_store import VectorStoreService
from app.retriever import Retriever
from app.llm import LLMService
from app.rag_pipeline import RAGPipeline
from config.settings import settings

# Page configuration
st.set_page.config(
    page_title="RAG Document Assistant",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded"
)
st.title(" RAG Document Assistant")
st.markdown("Ask questions about your documents using RAG
        (Ollama / OpenAI)")

# Sidebar - Configuration
st.sidebar.header(" Configuration")
llm_model = st.sidebar.selectbox(
    "Select LLM Model",
    ["ollama", "openai"],
    index=0)
)
if llm_model == "ollama":
    model_name = st.sidebar.text_input("Ollama Model",
                                       "llama3")
else:
    model_name = st.sidebar.text_input("OpenAI Model",
                                       "gpt-3.5-turbo")
top_k = st.sidebar.slider("Top K Results", 1, 10, 4)
temperature = st.sidebar.slider("Temperature", 0.0, 1.0, 0.2, 0.1)

# Initialize services
embedding_service = EmbeddingService()
vector_store = VectorStoreService()
retriever = Retriever(vector_store, embedding_service)
llm_service = LLMService(model_name=medel_name, provider=llm_model,
                         temperature=temperature)
rag_pipeline = RAGPipeline(retriever, llm_service)
```

4. Document Upload Section

```
st.header(" Upload Documents")
uploaded_files = st.file_uploader(
    "Upload PDF, TXT or DOCX files",
    type=["pdf", "txt", "docx"],
    accept_multiple_files=True
)
if uploaded_files:
    if st.button(" Index Documents"):
        with st.spinner("Processing documents..."):
            all_docs = []
            for file in uploaded_files:
                # Save uploaded file to temp location
                with tempfile.NamedTemporaryFile(delete=False, suffix=Path(file.name).suffix) as tmp:
                    tmp.write(file.getvalue())
                    tmp_path = tmp.name

                # Load and split documents
                docs = load_documents(tmp_path)
                chunks = split_documents(docs)
                all_docs.extend(chunks)

            # Store in Qdrant
            vector_store.add_documents(all_docs)
            st.success(f" Successfully indexed {len(all_docs)} chunks!")
```

5. Chat Interface (Question Input and Response)

```
st.header(" Ask a Question")
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display chat history
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# User input
query = st.chat_input("Type your question here...")
if query:
    st.session_state.messages.append({"role": "user", "content": query})
    with st.chat_message("user"):
        st.markdown(query)
    with st.chat_message("assistant"):
        with st.spinner("Generating answer..."):
            result = rag_pipeline.ask(query, k=top_k)
            answer = result.get("answer", "No answer found.")
            sources = result.get("sources", [])
            st.markdown(answer)

            # Show source documents
            if sources:
                st.markdown("### Source Documents")
                for i, src in enumerate(sources, 1):
                    with st.expander(f"Source {i}:
                        {src.get('metadata', {}).get('source','Unknown')}}":
                            st.markdown(src.get("content", "")[:500] + "...")
    st.session_state.messages.append({"role": "assistant", "content": answer})
```

6. Sample UI Layout



7. Running the Streamlit App

1. Install dependencies (if not installed)
   ```
   pip install -r requirements.txt
   ```
2. Run the Streamlit application
   ```
   streamlit run app/streamlit_app.py
   ```
3. Open the application in browser
   ```
   http://localhost:8501
   ```

8. Customization Ideas

- Add user authentication.
- Support different file types (CSV, Excel, etc).
- Add chat history persistence (database).
- Show document thumbnails and metadata.
- Add option to clear chat and re-index documents.
- Use custom CSS for better UI.
- Deploy on cloud (Streamlit Cloud / AWS / Docker).

9. Key Points

- Uses all previously created modules.
- Supports document upload and indexing.
- Interactive chat interface.
- Displays source documents with citations.
- Easy to run and deploy.
- Can be extended with more features.

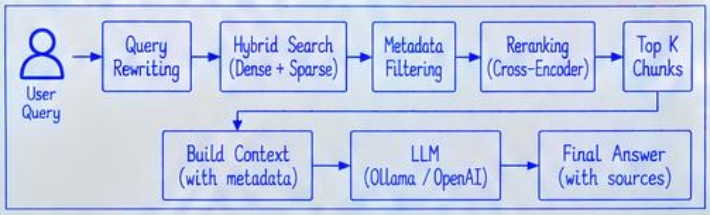
Rahul Yadav

Advanced RAG Features - Hybrid Search, Metadata Filtering, Reranking & Query Rewriting

1. Overview

- Advanced techniques to improve RAG performance.
- Hybrid search (dense + sparse) for better accuracy.
- Metadata filtering to retrieve relevant documents.
- Reranking to get most relevant chunks.
- Query rewriting to handle ambiguous questions.
- Helps reduce hallucinations and improves answer quality.

2. Advanced RAG Architecture



3. Hybrid Search (Dense + Sparse) - vector_store.py

```python
from qdrant_client import QdrantClient, models
from typing import List

class HybridVectorStore:
    def __init__(self, url: str, collection_name: str):
        self.client = QdrantClient(url=url)
        self.collection_name = collection_name

    def hybrid_search(self, query: str, query_embedding: List[float],
                      top_k: int = 5, alpha: float = 0.5):
        """Hybrid search using dense (vector) + sparse (BM25)"""
        returrn self.client.query_points(
            collection_name=self.collection_name,
            query=models.QueryVector(
                vector=query_embedding,
            ),
            query_filter=None,
            limit=top_k,
            with_payload=True,
            with_vectors=True,
            search_params=models.SearchParams(
                hnsw_ef=128, exact=False
            ),
            # Use score fusion (dense + sparse)
            vectors=['dense', 'sparse'],
            score_fusion=models.FusionQuery(
                fusion=models.Fusion.RRF,  # Reciprocal Rank Fusion
            )
        )
```

4. Metadata Filtering - vector_store.py

```python
def search_with_filter(self, query_embedding: List[float],
                       top_k: int = 5,
                       file_type: str = None,
                       source: str = None,
                       metadata: dict = None):
    """Search with metadata filters"""
    filter_conditions = []
    if file_type:
        filter_conditions.append(
            models.FieldCondition(
                key='file_type',
                match=models.MatchValue(value=file_type)
            )
        )
    if source:
        filter_conditions.append(
            models.FieldCondition(
                key='source',
                match=models.MatchValue(value=source)
            )
        )
    if metadata:
        for key, value in metadata.items():
            filter_conditions.append(
                models.FieldCondition(
                    key=key,
                    match=models.MatchValue(value=value)
                )
            )
    query_filter = models.Filter(must=filter_conditions) if filter_conditions else None

    return self.client.query_points(
        collection_name=self.collection_name,
        query_vector=query_embedding,
        query_filter=query_filter,
        limit=top_k,
        with_payload=True
    )
```

5. Reranking (Cross-Encoder) - reranker.py

```python
from sentence_transformers import CrossEncoder
from typing import List, Tuple

class Reranker:
    def __init__(self, model_name: str ="cross-encoder/ms-marco-MiniLM-L-6-v2"):
        self.model = CrossEncoder(model_name)

    def rerank(self, query: str, docs: List[dict], top_k: int = 5):
        """Rerank retrieved documents using cross-encoder"""
        pairs = [[query, doc['content']] for doc in docs]
        scores = self.model.predict(pairs)

        # Combine docs with scores
        ranked_docs = sorted(
            zip(docs, scores),
            key=lambda x: x[1],
            reverse=True
        )

        return [doc for doc, score in ranked_docs[:top_k]]
```

6. Query Rewriting - query_rewriter.py

```python
from langchain.llms import OpenAI

class QueryRewriter:
    def __init__(self, model='gpt-3.5-turbo"):
        self.llm = OpenAI(model_name=model, temperature=0)

    def rewrite(self, query: str) -> str:
        prompt = f"""Rewrite the following user question into a clear,
        standalone question for document retrieval. Do not answer the question.

        Original Question: {query}
        Rewritten Question:"""
        response = self.llm.predict(prompt)
        return response.strip()
```

7. Integrate Everything - rag_pipeline.py

```python
from app.vector_store import HybridVectorStore
from app.reranker import Reranker
from app.query_rewriter import QueryRewriter
from app.llm import generate_answer

class AdvancedRAGPipeline:
    def __init__(self, settings):
        self.vector_store = HybridVectorStore(settings.QDRANT_URL,
                                              settings.QDRANT_COLLECTION)
        self.reranker = Reranker()
        self.rewriter = QueryRewriter()
        self.top_k = settings.TOP_K

    def retrieve(self, query: str, filters: dict = None):
        # 1. Rewrite query
        rewritten_query = self.rewriter.rewrite(query)
        # 2. Generate dense embedding
        query_embedding = self.vector_store.get_embedding(rewritton_query)
        # 3. Hybrid search with metadata filter
        docs = self.vector_store.search_with_filter(
            query_embedding, top_k=self.top_k * 2, **(filters or {})
        )
        # 4. Rerank results
        reranked_docs = self.reranker.rerank(rewritten_query, docs, top_k=self.top_k)
        return reranked_docs, rewritten_query
```

8. Example Usage

```python
if __name__ == "__main__":
    from app.config.settings import load_settings
    settings = load_settings()
    pipeline = AdvancedRAGPipeline(settings)

    query = "What is RAG and how does it work?"
    filters = {"file_type": "pdf"}

    docs, rewritten = pipeline.retrieve(query, filters)

    # Build context
    context = "\n\n".join([doc["content"] for doc in docs])

    # Generate answer
    answer = generate_answer(query, context)

    print(f"Original Query: {query}")
    print(f"Rewritten Query: {rewritten}")
    print("\nAnswer=", answer)
    print("\nSources:")
    for i, doc in enumerate(docs):
        print(f"[{i+1} [doc['metadata'].get('source', 'Unknown')}"
              f" (Score: {round(doc['score'], 4)})")
```

9. Configuration (settings.py)

```python
# Advanced RAG settings
HYBRID_ALPHA = 0.5        # Balance between dense and sparse search
RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"
QUERY_REWRITE_MODEL = "gpt-3.5-turbo"  # or llama3 (Ollama)
TOP_K = 5
ENABLE_QUERY_REWRITE = True
ENABLE_RERANKING = True
```

10. Key Points

- Hybrid search improves retrieval accuracy (dense + sparse).
- Metadata filtering helps in domain specific search.
- Reranking using cross-encoder gives better relevant results.
- Query rewriting handles ambiguous or incomplete questions.
- Combine all techniques for production-ready RAG.

Rahul Yadav

RAG Evaluation & Testing - RAGAS, Faithfulness, Context Precision, Recall & Automated Testing

1. Overview

- Evaluation is important to measure the quality of your RAG system.
- Helps to check if answers are accurate, relevant and grounded in the retrieved context.
- RAGAS is a popular open-source library for evaluating RAG.
- We will use metrics like Faithfulness, Answer Relevancy, Context Precision, Context Recall.
- Also show how to create a test dataset and run automated evaluation.

2. Evaluation Metrics (RAGAS)

Metric	What it Measures	Ideal Value	Description
Faithfulness	Is the answer grounded in the retrieved context?	Close to 1.0	Measures hallucination. Higher is better.
Answer Relevancy	Is the answer relevant to the question?	Close to 1.0	Checks if the answer addresses the question.
Context Precision	How many retrieved chunks are relevant?	Close to 1.0	Measures precision of retrieved context.
Context Recall	Did we retrieve all relevant information?	Close to 1.0	Measures recall of retrieved context.

3. Install Required Libraries

```
# Install required libraries
pip install ragas datasets pandas tqdm
```

4. Prepare Evaluation Dataset - eval_dataset.py

```
import json
from datasets import Dataset

# Sample evaluation dataset
data = [
    {
        "question": "What is RAG?",
        "answer": "RAG (Retrieval Augmented Generation) combines
         information retrieval with text generation to provide
         accurate answers using external documents."
    },
    {
        "question": "How does vector database work?",
        "answer": "A vector database stores embeddings and allows
         similarity search to retrieve relevant documents."
    },
    {
        "question": "What is Qdrant?",
        "answer": "Qdrant is an open-source vector database
         optimized for high-dimensional embeddings."
    }
]
# Convert to HuggingFace Dataset
dataset = Dataset.from_list(data)
dataset.to_json("eval_dataset.json")
print("Evaluation dataset saved to eval_dataset.json")
```

5. Run Evaluation with RAGAS - evaluate.py

```
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
)
from app.rag_pipeline import RAGPipeline
from app.config_settings import load_settings
import pandas as pd

# Load evaluation dataset
dataset = Dataset.from_json("eval_dataset.json")
settings = load_settings()

# Initialize RAG pipeline
rag = RAGPipeline(settings)

# Generate answers for all questions
results = []
for item in dataset:
    question = item["question"]
    answer, sources = rag.generate_answer(question)
    results.append({
        "question": question,
        "answer": answer,
        "contexts": [doc["content"] for doc in sources],
        "ground_truth": item["answer"]
    })

# Convert to dataset for evaluation
eval_dataset = Dataset.from_list(results)

# Run RAGAS evaluation
metrics = [faithfulness, answer_relevancy, context_precision, context_recall]
score = evaluate(eval_dataset, metrics=metrics)

# Convert results to DataFrame
df = score.to_pandas()
print("\n=== RAGAS Evaluation Results ===")
print(df)
df.to_csv("evaluation_results.csv", index=False)
print("\nResults saved to evaluation_results.csv")
```

6. Custom Evaluation (Optional Metrics)

```
from sentence_transformers import SentenceTransformer, util
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")
def semantic_similarity(text1: str, text2: str) -> float:
    embeddings = model.encode([text1, text2])
    score = util.cos_sim(embeddings[0], embeddings[1]).item()
    return float(score)

# Example: compare generated answer with ground truth
generated_answer = "RAG uses retrieval and generation..."
ground_truth = "RAG combines information retrieval with text
generation..."
similarity = semantic_similarity(generated_answer, ground_truth)
print(f"Semantic Similarity: {similarity:.4f}")
```

7. Sample Output

```
# Console Output (evaluation results)
```

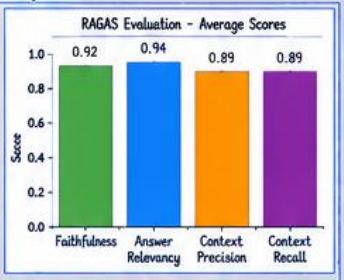
question	faithfulness	answer_relevancy	context_precision	context_recall
What is RAG?	0.92	0.95	0.90	0.88
How does vector DB work?	0.89	0.91	0.85	0.87
What is Qdrant?	0.94	0.96	0.92	0.91
Average	**0.92**	**0.94**	**0.89**	**0.89**

8. Visualize Results - Plot Graphs

```
import matplotlib.pyplot as plt
import pandas as pd
df = pd.read_csv("evaluation_results.csv")
metrics = ["faithfulness", "answer_relevancy",
           "context_precision", "context_recall"]

plt.figure(figsize=(8, 4))
bars = plt.bar(metrics, df[metrics].mean(),
        color=["#4CAF50", "#2196F3",
               "#FF9800", "#9C27B0"])
plt.title("RAGAS Evaluation - Average Scores")
plt.ylabel("Score")
plt.ylim(0, 1.0)

for i, bar in enumerate(bars):
    plt.text(i, bar.get_height() + 0.02, f"{bar.get_height():.2f}",
             ha='center")
plt.tight_layout()
plt.savefig("evaluation_results.png")
plt.show()
```



9. Automated Testing - test_rag.py

```
import pytest
from app.rag_pipeline import RAGPipeline
from app.config.settings import load_settings

def test_rag_answer():
    settings = load_settings()
    rag = RAGPipeline(settings)
    question = "What is RAG?"
    answer, sources = rag.generate_answer(question)
    assert len(answer) > 10
    assert len(sources) > 0
    assert "RAG" in answer or "retrieval" in answer.lower()
```

Run Tests

```
# Run all tests
pytest test_rag.py

# Run with verbose output
pytest -v test_rag.py
```

10. Key Points

- Use RAGAS to evaluate answer quality, groundedness and relevance.
- Check multiple metrics: faithfulness, answer relevancy, context precision and recall.
- Create a good evaluation dataset with diverse questions.
- Automate testing to ensure consistent performance.
- Visualize results to analyze improvements.
- Use these metrics to compare different models, chunking strategies and retrieval methods.
- Helps to reduce hallucinations and improve RAG performance.

Rahul Yadav

Complete End-to-End RAG Project – Final Code Integration, Execution, Testing & GitHub Deployment

1. Project Recap

- We built a complete RAG (Retrieval Augmented Generation) project.
- Used document ingestion, chunking, embeddings and Qdrant vector DB.
- Integrated with LLMs (Ollama / OpenAI) for answering questions.
- Built a Streamlit UI for easy interaction.
- Added advanced features like hybrid search, reranking, metadata filtering
- Evaluated using RAGAS and added automated tests.
- Containerized using Docker & Docker Compose.
- Finally prepared the project for GitHub deployment.

2. Final Project Structure

```
rag-project/
├── app/
│   ├── __init__.py
│   ├── main.py                 # CLI application
│   ├── streamlit_app.py        # Web UI
│   ├── rag_pipeline.py         # Complete pipeline
│   ├── config_settings.py      # Configuration
│   ├── evaluation.py           # RAGAS evaluation
│   ├── test_rag.py             # Automated tests
│   ├── ingestion.py
│   ├── chunking.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── llm.py
│   ├── reranker.py
│   ├── query_rewriter.py
│   └── generate_answer.py
├── data/
│   ├── documents/              # Put your PDF, TXT, DOCX files here
│   └── outputs/                # Generated outputs (optional)
├── requirements.txt            # Python dependencies
├── .env                        # Environment variables
├── README.md                   # Project documentation
├── Dockerfile                  # Docker image for app
├── docker-compose.yml          # Run app + qdrant + ollama
├── .dockerignore               # Exclude unnecessary files
├── eval_dataset.json           # Evaluation dataset
└── evaluation_results.csv      # Evaluation output
```

3. Complete RAG Pipeline Code (main.py)

```python
from app.rag_pipeline import RAGPipeline
from app.config_settings import load_settings

def main():
    # Load settings
    settings = load_settings()

    # Initialize pipeline
    pipeline = RAGPipeline(settings)
    print("RAG Pipeline Initialized Successfully!")

    while True:
        query = input("\nEnter your question (or 'quit'): ")
        if query.lower() in ['quit', 'exit']:
            print("Thank you! Exiting...")
            break
        result = pipeline.generate_answer(query)
        print("\nAnswer:", result["answer"])
        if result.get("sources"):
            print("\nSources:")
            for i, src in enumerate(result["sources"]):
                print(f"[{i+1}] {src['source']} (Score: {src['score']:.4f})")


if __name__ == "__main__":
    main()
```

4. Build and Run the Complete Project (Docker)

1. Build and start all services

```
docker-compose up --build -d
```

Builds the app, qdrant and ollama containers

2. View running containers

```
docker-compose ps
```

3. View logs

```
docker-compose logs -f
```

4. Stop all services

```
docker-compose down
```

5. Run Locally (Without Docker)

1. Create virtual environment

```
python -m venv .venv
```

Windows: .venv\Scripts\activate
Linux/Mac: source .venv/bin/activate

2. Install dependencies

```
pip install -r requirements.txt
```

3. Set environment variables

```
cp .env.example .env    # Update with your API keys
```

4. Run the application
   - CLI Version: `python main.py`
   - Streamlit UI: `streamlit run app/streamlit_app.py`

6. Testing the Project

1. Run automated tests (pytest)

```
pytest test_rag.py
```

2. Run with verbose output

```
pytest -v test_rag.py
```

3. Run evaluation (RAGAS)

```
python app/evaluation.py
```

4. Expected output

```
=== RAGAS Evaluation Results ===
Faithfulness:      0.92
Answer Relevancy:  0.94
Context Precision: 0.89
Context Recall:    0.89
```

7. Deploy to GitHub

1. Initialize Git repository

```
git init
```

2. Add files

```
git add .
```

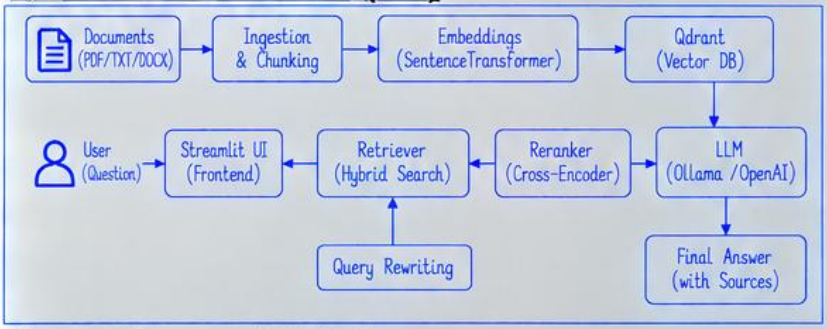
3. Commit changes

```
git commit -m "Initial commit - Complete RAG Project"
```

4. Create GitHub repository
   - Go to github.com
   - Create a new repository (e.g. rag-project)
   - Copy the repository URL
5. Push code to GitHub

```
git remote add origin https://github.com/username/rag-project.git
git push -u origin main
```

8. End-to-End Architecture (Final)



9. Sample End-to-End Output

```
$ python main.py
==========================================
RAG Pipeline Initialized Successfully!
Enter your question (or 'quit'): What is RAG?

Answer:
RAG (Retrieval Augmented Generation) is a technique
that combines information retrieval with text generation
to provide accurate and context-aware answers using
external documents.
Sources:
[1] rag_guide.pdf (Score: 0.9123)
[2] ai_basics.pdf (Score: 0.8734)

Enter your question (or 'quit'): quit
Thank you! Exiting...
```

10. Key Takeaways & Next Steps

Key Takeaways

- You learned how to build a complete RAG project from scratch.
- Understood document ingestion, chunking and embeddings.
- Used Qdrant for efficient vector storage and search.
- Integrated LLMs (Ollama / OpenAI) for answer generation.
- Built a user friendly Streamlit UI.
- Implemented advanced features like hybrid search, reranking and metadata filtering.
- Evaluated using RAGAS and added automated tests.
- Containerized the application using Docker.
- Prepared the project for GitHub deployment.

Next Steps / Enhancements

- Add user authentication and multi-user support.
- Support more document types (CSV, Excel, Markdown).
- Add chat history persistence (database).
- Deploy on cloud (AWS / GCP / Streamlit Cloud).
- Add monitoring and logging (e.g., LangSmith).
- Support multiple LLMs (Llama3, Claude, Gemini).
- Implement re-ranking with more advanced models.
- Add API endpoints using FastAPI.
- Build a production-ready UI with better design.

Useful Resources

- LangChain Documentation
- LlamaIndex Documentation
- Qdrant Documentation
- RAGAS Documentation
- Streamlit Documentation
- Ollama Documentation
- OpenAI Documentation
- HuggingFace Documentation
- Docker Documentation
- GitHub Documentation

Rahul Yadav